Cryptography

Stream Ciphers

Cristhian Iza

School of Mathematical and Computational Sciences
Yachay Tech University

August 26, 2026

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Learning Objectives

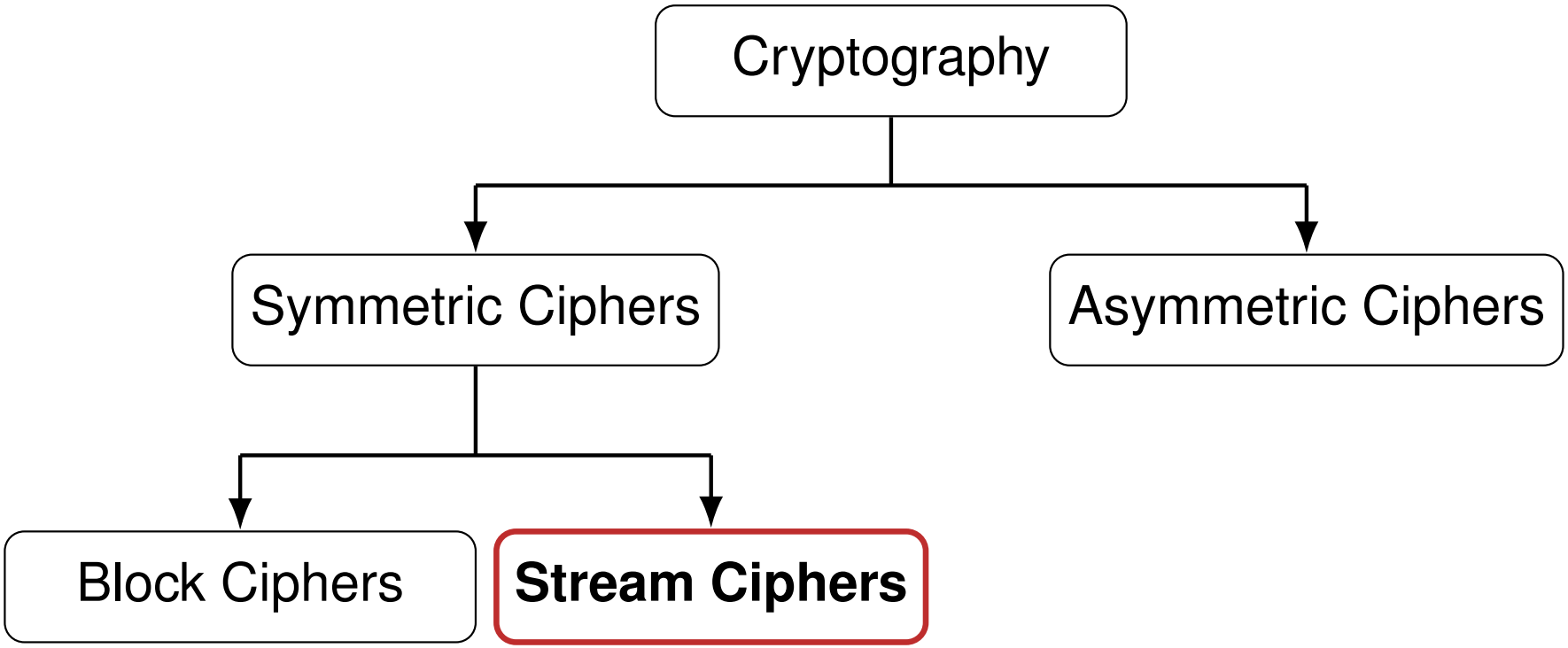
By the end of this chapter, students should be able to:

- distinguish stream ciphers from block ciphers;
- explain encryption and decryption using XOR;
- differentiate TRNGs, PRNGs, and CSPRNGs;
- explain why the One-Time Pad provides perfect secrecy;
- analyze the operation and weakness of an LFSR;
- describe the design principles of Trivium.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Introduction

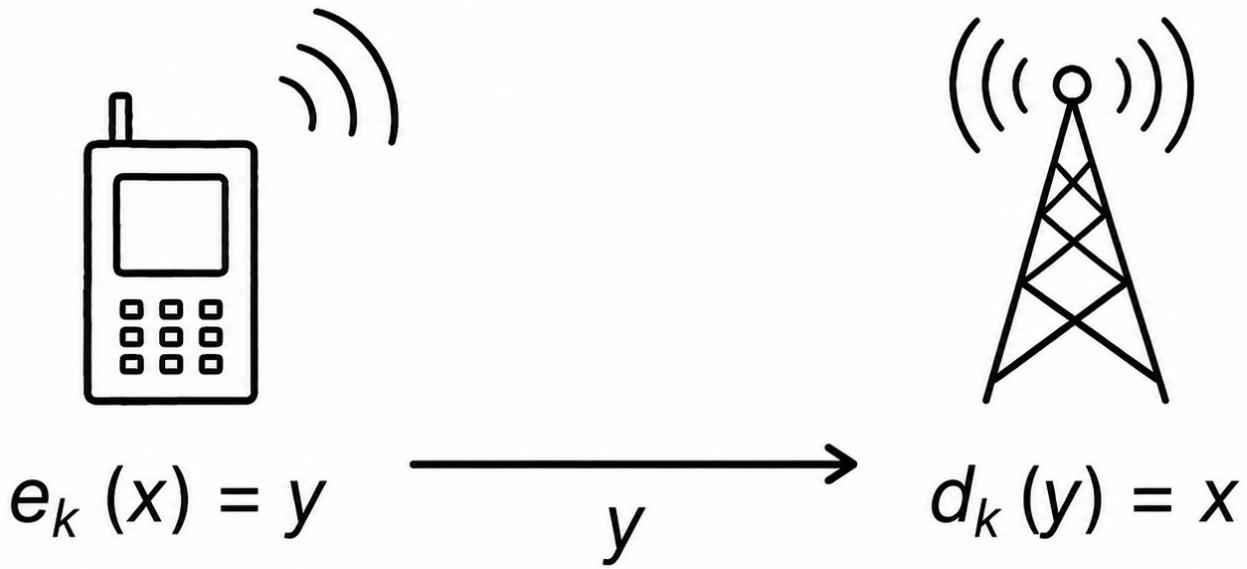
Where Do Stream Ciphers Fit?



Stream ciphers encrypt a message as a sequence of small units, typically bits.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Bitwise Stream Encryption



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Stream Cipher

Definition

- A stream cipher encrypts bits individually.

Stream Cipher vs. Block Cipher

Stream Cipher

- Processes plaintext incrementally.
- Combines plaintext with a keystream.
- Often attractive for low-latency or constrained systems.

Block Cipher

- Processes fixed-size blocks.
- Each output block depends on the complete input block.
- AES uses 128-bit blocks.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Basic Stream-Cipher Model

Let

$$x_i, y_i, s_i \in \{0, 1\},$$

where x_i is plaintext, y_i is ciphertext, and s_i is a keystream bit.

Encryption

$$y_i = e(x_i)$$

$$y_i = x_i + s_i \pmod 2$$

Decryption

$$x_i = d(y_i)$$

$$x_i = y_i + s_i \pmod 2$$

MATHEMATICAL AND COMPUTATIONAL SCIENCES

Basic Stream-Cipher Model

Question

Why are encryption and decryption the same operation?

Encryption

$$y_i = x_i + s_i \pmod 2$$

Decryption

$$x_i = y_i + s_i \pmod 2$$

Why Are Encryption and Decryption Identical?

Starting from

$$d(y_i) \equiv y_i - s_i \pmod 2,$$

since subtraction and addition are equivalent modulo 2,

$$y_i - s_i \equiv y_i + s_i$$

Therefore, using the same keystream bit,

$$\begin{aligned} d(y_i) &\equiv \\ &\equiv \\ &\equiv \end{aligned}$$

Question

Why are encryption and decryption the same operation?

$$x_i,\ y_i,\ s_i \in \mathbb{Z}_2 = \{0, 1\}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Closer Look at mod 2 add

x_i	s_i	$y_i = x_i + s_i \mod 2$
0	0	
0	1	
1	0	
1	1	

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Closer Look at mod 2 add

$$\boxed{y_i = x_i + s_i \mod 2}$$

If s_i is uniformly random,

$$\Pr(y_i = 0) = \Pr(y_i = 1) = \frac{1}{2},$$

independently of the plaintext bit.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Stream Cipher: Encryption and Decryption of ASCII A

Encryption

$$\begin{aligned} x_1 &= 1000001 \\ s_1 &= 0101101 \\ \hline y_1 = x_1 \oplus s_1 &= \end{aligned}$$

Decryption

$$\begin{aligned} y_1 &= \\ s_1 &= 0101101 \\ \hline x_1 = y_1 \oplus s_1 &= \end{aligned}$$

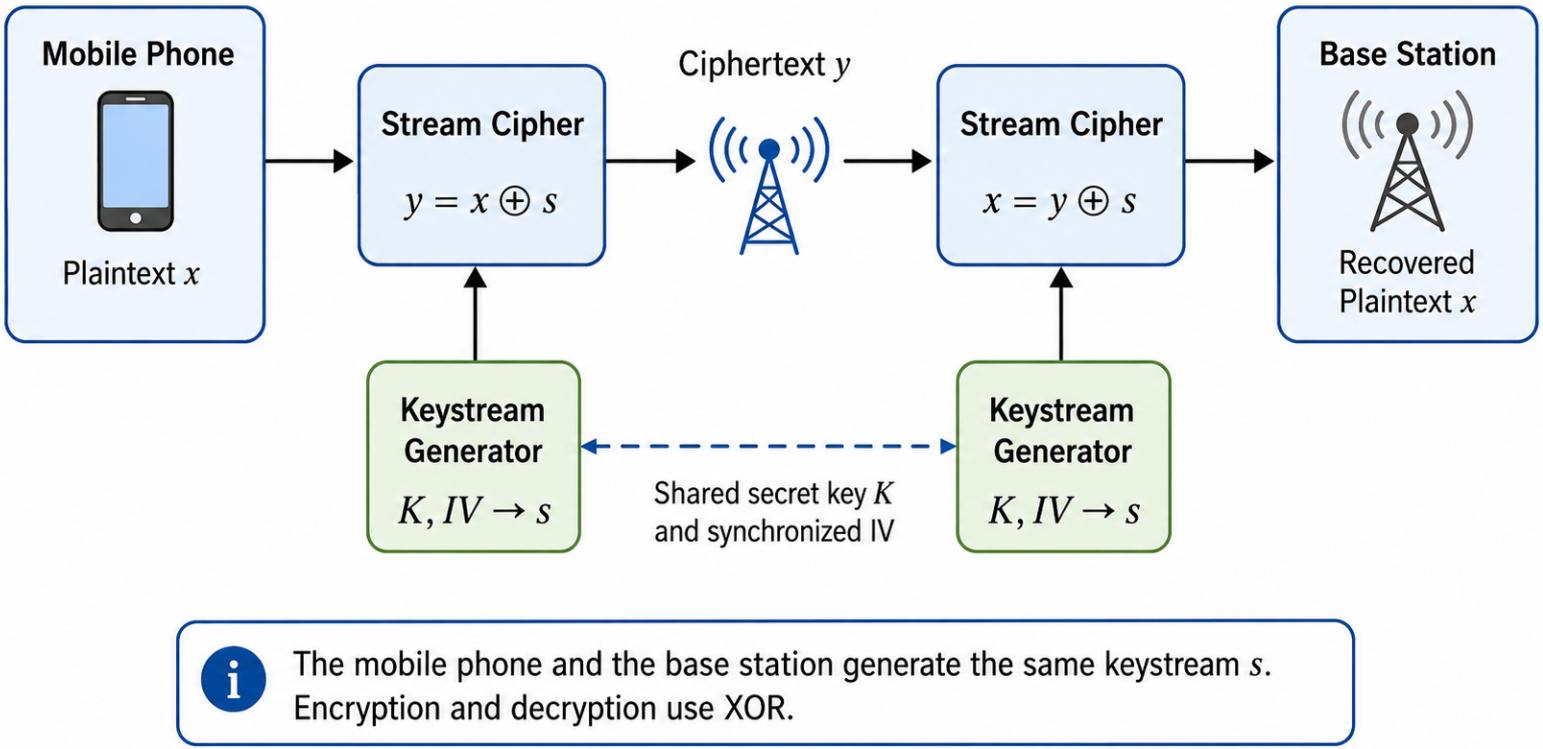
Key Idea

The same keystream s_1 is used for both encryption and decryption.

$$(x_1 \oplus s_1) \oplus s_1 = x_1$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Bitwise Stream Encryption



The mobile phone and the base station generate the same keystream s.
Encryption and decryption use XOR.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

The Central Security Problem

Encryption itself is simple:

$$y_i = x_i \oplus s_i.$$

The difficult problem is generating the sequence

$$s_0, s_1, s_2, \ldots$$

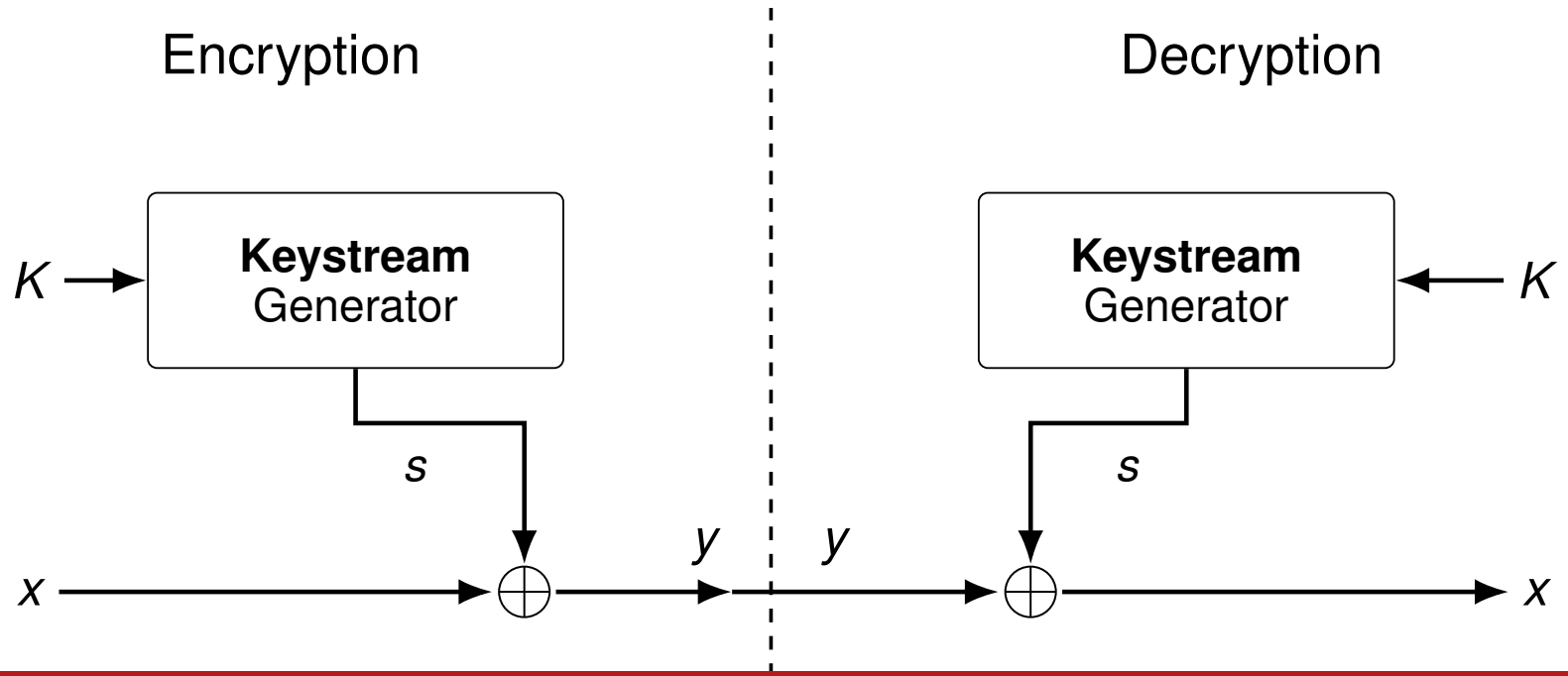
with suitable cryptographic properties.

Security principle

A stream cipher is only as secure as its keystream generator.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

The Central Security Problem



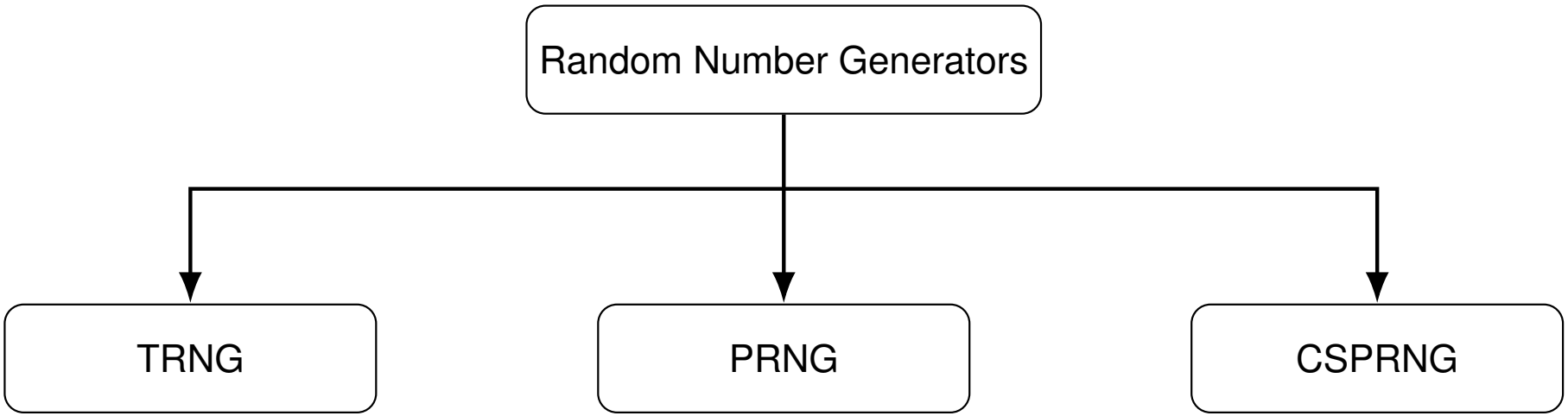
Question

How do we generate the key stream bits s_i?

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RNG's

Random Number Generators (RNG)



The distinction is not merely statistical: **predictability** is the cryptographic issue.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

True Random Number Generator (TRNG)

A TRNG derives randomness from a physical entropy source.

- coin flipping;;
- mouse movement;
- key stroke timing;
- other nondeterministic physical processes.

Key Point

A TRNG output is not reproducible. This is useful for generating keys, nonces, and seeds, but unsuitable for directly generating a shared synchronous keystream.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Pseudorandom Number Generator (PRNG)

A PRNG is deterministic and starts from an initial seed:

$$s_0 = \text{seed}, \qquad s_{i+1} = f(s_i).$$

A common example is the linear congruential generator:

$$s_{i+1} \equiv as_i + b \pmod{m}.$$

Important

A sequence may have good statistical properties and still be predictable. Therefore, an ordinary PRNG is not automatically suitable for cryptography.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Random Numbers in C: `rand()`

C provides the function

`rand()`

to generate **pseudo-random integers**.

Basic usage

```
#include <stdlib.h>
#include <time.h>

srand(time(NULL));
int r = rand();
```

Cryptographic limitation

`rand()` is deterministic and predictable; therefore, it should not be used to generate cryptographic keys or keystreams.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

From Seed to Pseudo-Random Numbers

A simple PRNG can be modeled by a **Linear Congruential Generator (LCG)**:

$$\boxed{X_{n+1} = (aX_n + c) \bmod m}$$

where

$$X_0 = \text{seed}.$$

Classical example

$$X_{n+1} = (1103515245X_n + 12345) \bmod 2^{31}$$

$$\text{seed} \longrightarrow X_1 \longrightarrow X_2 \longrightarrow X_3 \longrightarrow \cdots$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Note: the C standard does not require `rand()` to use this specific formula; it is a classical PRNG model.

CSPRNG: Cryptographically Secure Pseudo-Random Number

A cryptographically secure PRNG must be **unpredictable**. Given

$$s_i, s_{i+1}, \ldots, s_{i+n-1},$$

it must be computationally infeasible to predict

$$s_{i+n}, s_{i+n+1}, \ldots$$

with useful advantage.

Key Point

Statistical randomness is necessary, but cryptographic unpredictability is the stronger requirement.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Why a Simple Linear PRNG Fails

Consider

$$S_{i+1} = AS_i + B \pmod{m}.$$

If an attacker recovers three consecutive states S_1, S_2, S_3, then

$$S_2 = AS_1 + B \pmod{m},$$

$$S_3 = AS_2 + B \pmod{m}.$$

This gives two linear equations in the unknowns A and B.

Consequence

Linearity can allow recovery of the generator parameters and prediction of the entire future keystream.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

One-Time Pad

Goal: build a perfect cipher

Definition

A cipher is "unconditionally secure" it it can not be broken even with infinite computing resources

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

One-Time Pad (OTP)

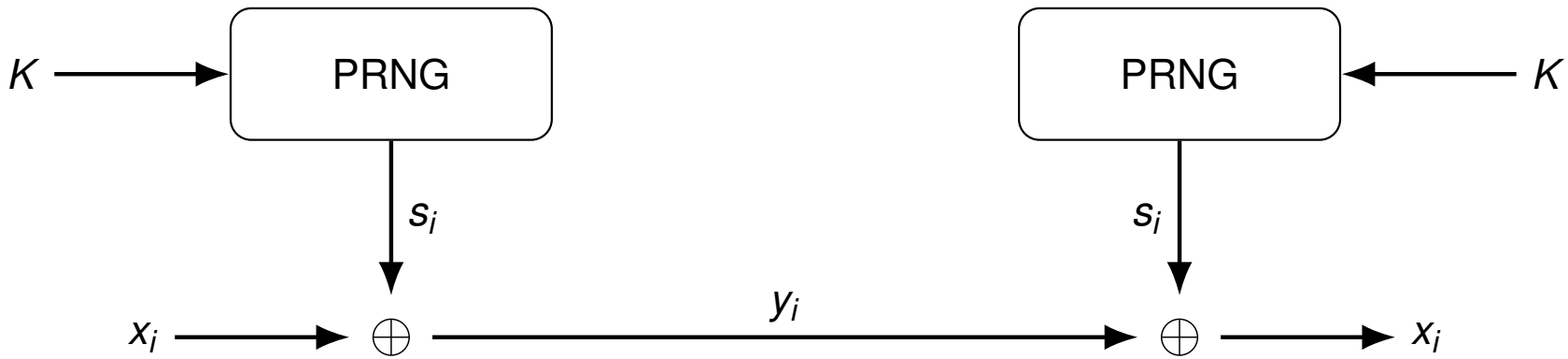
Definition

The **One-Time Pad (OTP)** is a stream cipher with the following properties:

- The keystream bits s_i are generated by a **TRNG**.
- Each keystream bit s_i is used **only once**.
- The keystream has the **same length** as the plaintext.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Stream Cipher: Block Diagram



$$y_i = x_i \oplus s_i \qquad x_i = y_i \oplus s_i$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

One-Time Pad (OTP)

The OTP uses a truly random key bit k_i for every plaintext bit:

$$y_i = x_i \oplus k_i, \qquad x_i = y_i \oplus k_i.$$

Requirements for perfect secrecy:

1. the key is truly random;
2. the key is as long as the message;
3. every key bit is used exactly once.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Why the OTP Is Unconditionally Secure

For a known ciphertext bit y_i:

$$y_i = x_i \oplus k_i.$$

If k_i is independent and uniformly distributed,

$$\Pr(x_i = 0 \mid y_i) = \Pr(x_i = 1 \mid y_i) = \frac{1}{2}.$$

Interpretation

The ciphertext provides no information that makes one plaintext bit more likely than the other.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

The Cost of Perfect Secrecy

For an n-bit message, the OTP requires an independent n-bit random key:

$$|K| = |M|.$$

- The key must be distributed securely.
- The key must be stored securely.
- The key must never be reused.

Practical limitation

Perfect secrecy is obtained at the cost of impractical key generation and distribution for most applications.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

From OTP to Practical Stream Ciphers

Practical stream ciphers replace the truly random OTP key with a pseudorandom keystream:

$$k \xrightarrow{\text{CSPRNG / keystream generator}} s_0, s_1, s_2, \ldots$$

Then

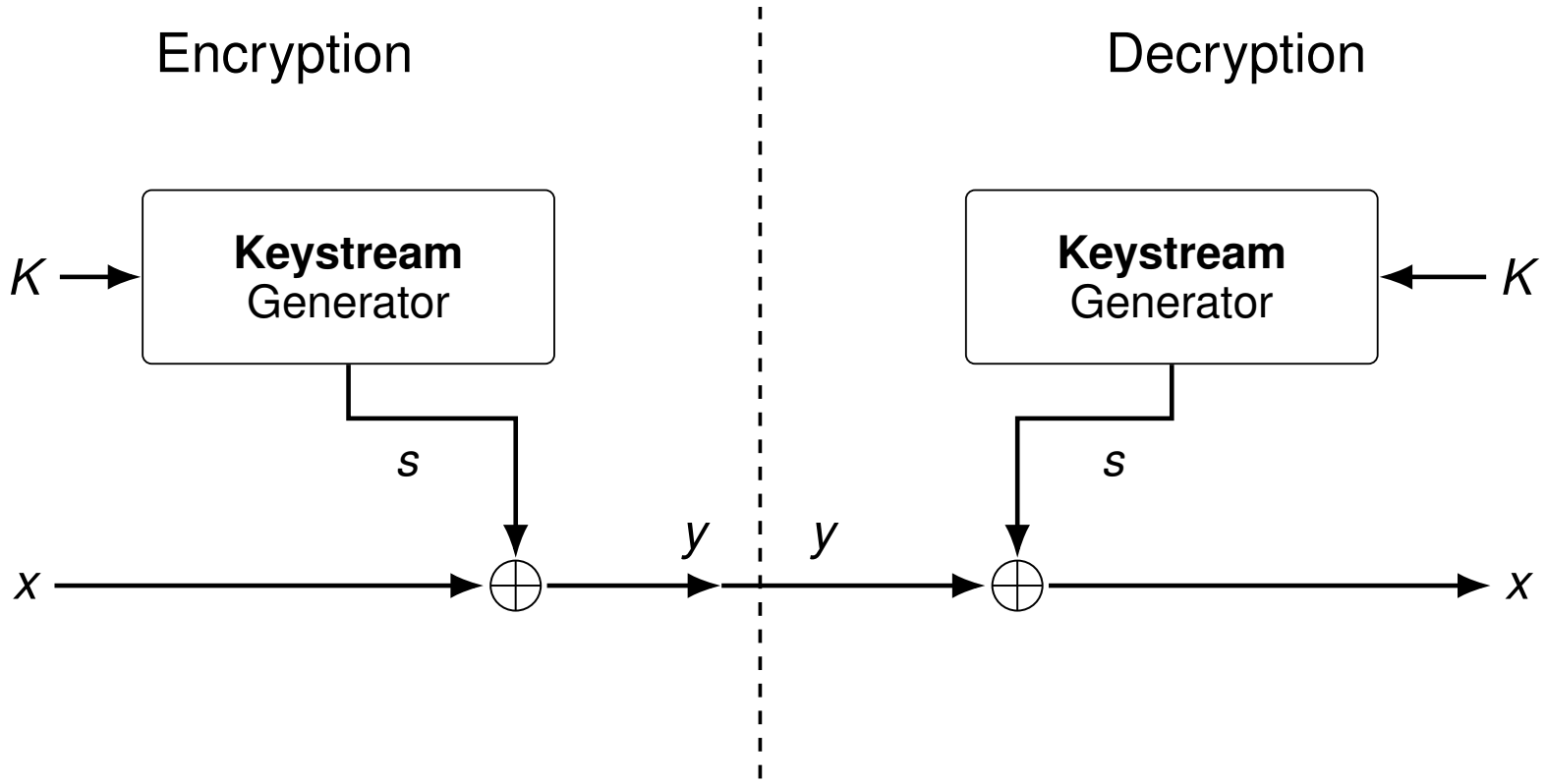
$$y_i = x_i \oplus s_i.$$

Trade-off

A short secret key replaces the message-length OTP key, but security becomes **computational**, not unconditional.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Stream Cipher: Encryption and Decryption



$$y = x \oplus s$$

$$x = y \oplus s$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

LFSRs

Linear Feedback Shift Registers

Goal

Stream cipher that is small in hardware (low power)

Linear Feedback Shift Register (LFSR) (cont.)

Basic Idea

An **LFSR** generates a sequence of bits using:

$$\text{shift register} + \text{XOR feedback}$$

$$[1 \;\; 0 \;\; 1 \;\; 1] \longrightarrow [0 \;\; 1 \;\; 0 \;\; 1] \longrightarrow \cdots$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Linear Feedback Shift Register (LFSR) (cont.)

How it works

- The register stores a fixed number of bits.
- At each step, the bits are shifted by one position.
- A new bit is computed by XORing selected bits:

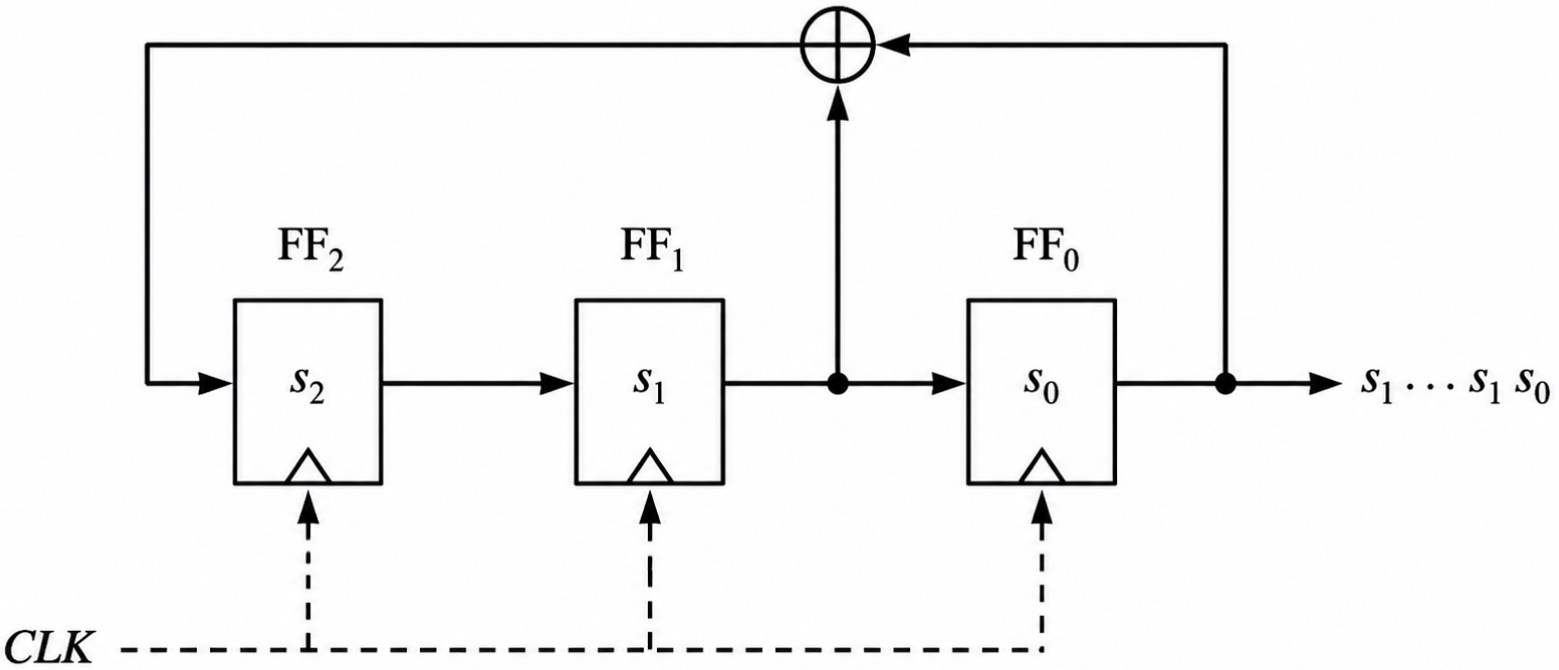
$$s_{\text{new}} = s_i \oplus s_j$$

Cryptographic limitation

A single LFSR is predictable because its operation is linear. Therefore, it is not secure enough for modern cryptography by itself.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

LFSR Constructed with Flip-Flops



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Linear Feedback Shift Registers

An LFSR consists of:

- m clocked storage elements (flip-flops);
- a shift operation;
- a feedback function formed by XOR of selected state bits.

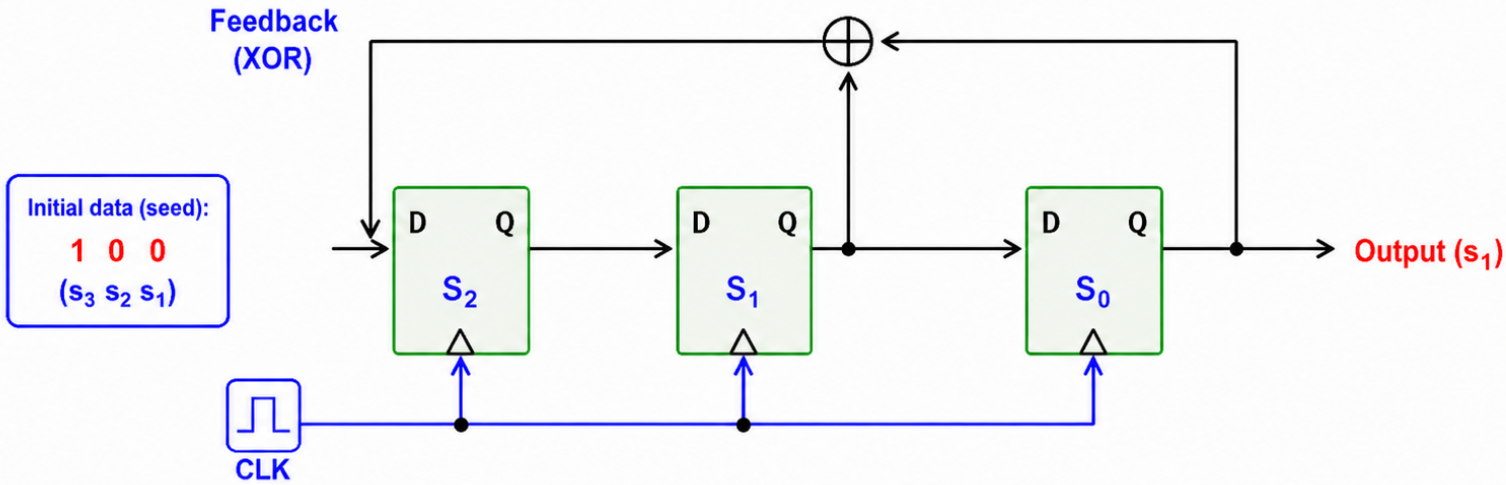
The number of flip-flops is the **degree** m.

Key Point

LFSRs are extremely efficient in hardware and generate long periodic sequences, but a single LFSR is cryptographically weak.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

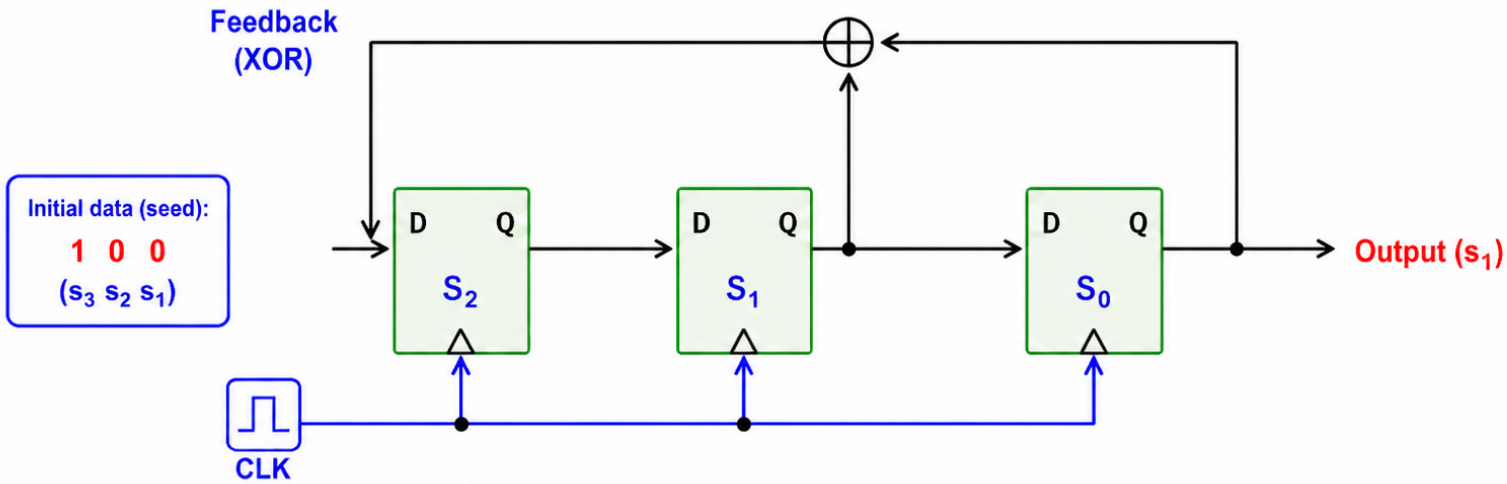
LFSR Constructed with Flip-Flops



Step	State (s_3 s_2 s_1)	Output (s_1)	Feedback bit $f = s_2 \oplus s_1$
0	1 0 0		
1	0 1 0		
2	1 0 1		
3	1 1 0		
4	1 1 1		
5	0 1 1		
6	0 0 1		
7	1 0 0		

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

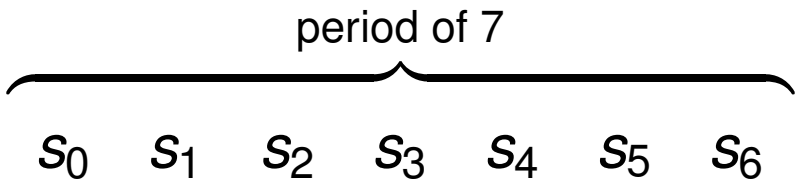
LFSR Constructed with Flip-Flops



Step	State (s_3 s_2 s_1)	Output (s_1)	Feedback bit $f = s_2 \oplus s_1$
0	1 0 0	0	0
1	0 1 0	0	1
2	1 0 1	1	1
3	1 1 0	0	1
4	1 1 1	1	0
5	0 1 1	1	0
6	0 0 1	1	1
7	1 0 0	0	0

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Period of 7



0 0 1 0 1 1 1

s_7 s_8 s_9 s_{10} s_{11} s_{12} s_{13}

0 0 1 0 1 1 1

s_{14} s_{15} s_{16} s_{17} s_{18} s_{19} s_{20}

0 0 1 0 1 1 1

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

LFSR Output Sequence

Assume the initial state bits are

$$s_0,\ s_1,\ s_2.$$

The following bits are generated recursively:

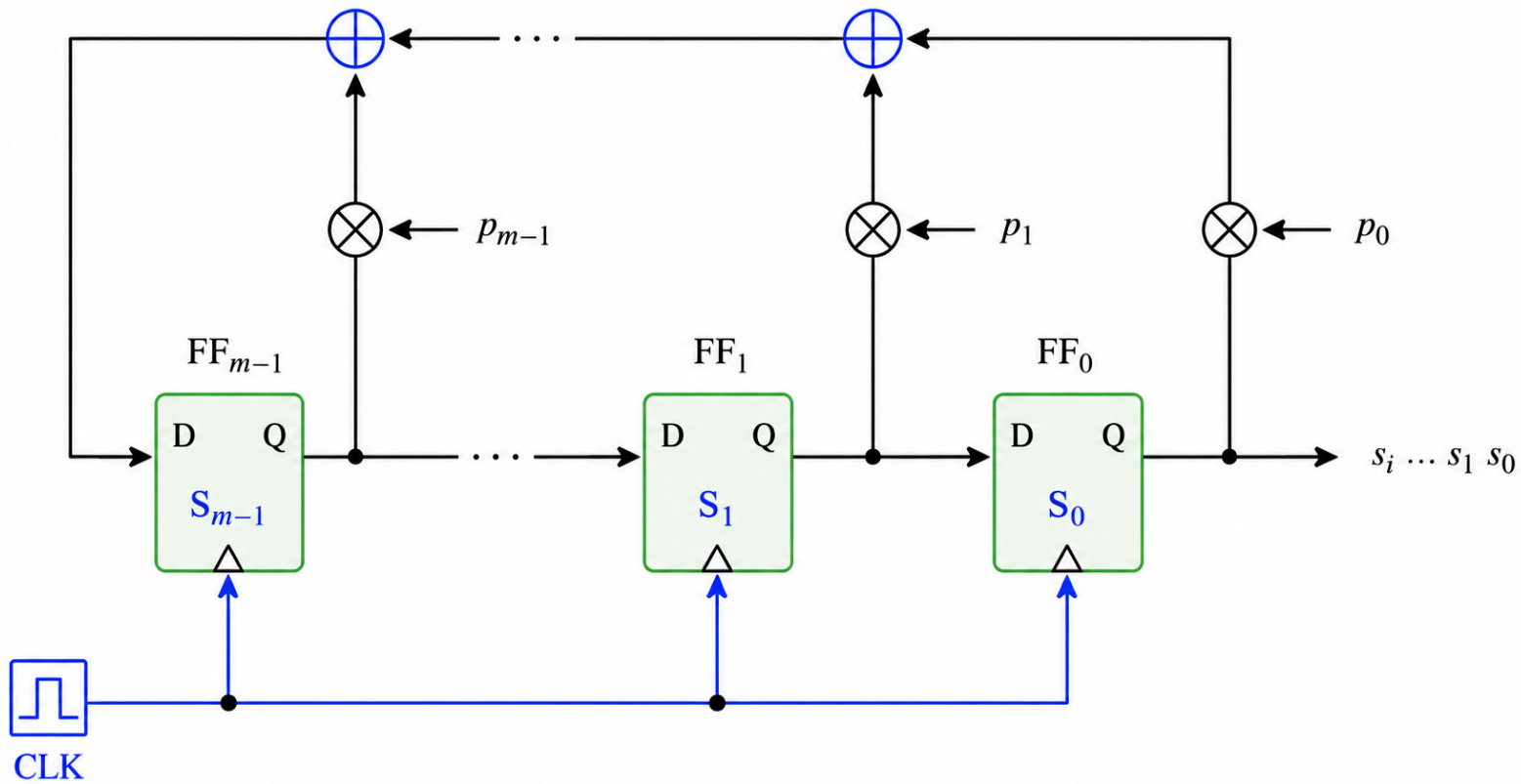
$$\begin{aligned} s_3 &\equiv s_1 + s_0 \pmod 2, \\ s_4 &\equiv s_2 + s_1 \pmod 2, \\ s_5 &\equiv s_3 + s_2 \pmod 2. \end{aligned}$$

General Recurrence

$$\boxed{s_{i+3} \equiv s_{i+1} + s_i \pmod 2} \qquad i = 0, 1, 2, \ldots$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

General LFSR with feedback coefficients p_i and initial values $s_{m-1}, \ldots, s_0$



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Mathematical Description of an LFSR (cont.)

Feedback coefficients

For an LFSR of degree m, each feedback coefficient

$$p_i \in \{0, 1\}$$

determines whether the output of a flip-flop contributes to the feedback:

- $p_i = 1$: feedback path is active.
- $p_i = 0$: feedback path is inactive.

The next bit is computed as the XOR-sum of the selected register outputs:

$$s_m \equiv s_{m-1}p_{m-1} + \cdots + s_1 p_1 + s_0 p_0 \pmod 2.$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Mathematical Description of an LFSR (cont.)

General recurrence

$$s_{m+i} \equiv \sum_{j=0}^{m-1} p_j \, s_{i+j} \pmod 2$$

$$s_i, p_j \in \{0, 1\}, \qquad i = 0, 1, 2, \ldots$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Mathematical Description of an LFSR

For an LFSR of degree m:

$$s_{i+m} \equiv \sum_{j=0}^{m-1} p_j s_{i+j} \pmod 2,$$

where

$$p_j \in \{0, 1\}$$

are feedback coefficients. The feedback polynomial can be written as

$$\boldsymbol{P(x) = x^m + p_{m-1}x^{m-1} + \cdots + p_1 x + p_0}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Example: LFSR of Degree 3

Consider

$$s_{i+3} = s_{i+1} + s_i \pmod 2$$

with initial state

$$(s_2, s_1, s_0) = (1, 0, 0).$$

The output sequence begins as

$$0010111\ 0010111\ 0010111\ldots$$

The period is

$$2^3 - 1 = 7.$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Maximum-Length LFSRs

An m-bit register has 2^m possible states. The all-zero state is absorbing, therefore a nonzero LFSR can have at most

$$\boxed{2^m - 1}$$

states in its cycle.

Maximum-length sequence

An LFSR achieves period $2^m - 1$ only for appropriate feedback polynomials. **Only certain feedback configuration $(p_{m-1}, \ldots, p + 0)$ yield maximum length sequences.**

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Primitive polynomials for maximum-length LFSRs

(0, 1, 2)
(0, 1, 3)
(0, 1, 4)
(0, 2, 5)
(0, 1, 6)
(0, 1, 7)
(0, 2, 3, 4, 8)
(0, 4, 9)
(0, 3, 10)
(0, 2, 11)
(0, 1, 4, 6, 12)
(0, 1, 3, 4, 13)
(0, 1, 3, 5, 14)
(0, 1, 15)
(0, 2, 3, 5, 16)
(0, 3, 17)
(0, 1, 2, 5, 18)
(0, 1, 2, 5, 19)
(0, 3, 20)
(0, 2, 21)
(0, 1, 22)
(0, 5, 23)
(0, 1, 3, 4, 5, 24)
(0, 3, 25)
(0, 1, 2, 6, 26)
(0, 1, 2, 5, 27)
(0, 3, 28)
(0, 2, 29)
(0, 1, 4, 6, 30)
(0, 3, 31)
(0, 1, 2, 3, 5, 7, 32)
(0, 1, 4, 6, 33)
(0, 1, 2, 5, 6, 7, 34)
(0, 2, 35)
(0, 1, 2, 4, 5, 6, 36)
(0, 1, 2, 3, 4, 5, 37)
(0, 1, 5, 6, 38)
(0, 4, 39)
(0, 3, 4, 5, 40)
(0, 3, 41)
(0, 1, 2, 3, 4, 5, 42)
(0, 3, 4, 6, 43)
(0, 2, 5, 6, 44)
(0, 1, 3, 4, 45)
(0, 1, 2, 3, 5, 8, 46)
(0, 5, 47)
(0, 1, 2, 4, 5, 7, 48)
(0, 4, 5, 6, 49)
(0, 2, 3, 4, 50)
(0, 1, 3, 6, 51)
(0, 3, 52)
(0, 1, 2, 6, 53)
(0, 2, 3, 4, 5, 6, 54)
(0, 1, 2, 6, 55)
(0, 2, 4, 7, 56)
(0, 2, 3, 5, 57)
(0, 1, 5, 6, 58)
(0, 1, 3, 4, 5, 6, 59)
(0, 1, 60)
(0, 1, 2, 5, 61)
(0, 3, 5, 6, 62)
(0, 1, 63)
(0, 1, 3, 4, 64)
(0, 1, 3, 4, 65)
(0, 2, 3, 5, 6, 8, 66)
(0, 1, 2, 5, 67)
(0, 1, 5, 7, 68)
(0, 2, 5, 6, 69)
(0, 1, 3, 5, 70)
(0, 1, 3, 5, 71)
(0, 1, 2, 3, 4, 6, 72)
(0, 2, 3, 4, 73)
(0, 3, 4, 7, 74)
(0, 1, 3, 6, 75)
(0, 2, 4, 5, 76)
(0, 2, 5, 6, 77)
(0, 1, 2, 7, 78)
(0, 2, 3, 4, 79)
(0, 1, 2, 3, 5, 7, 80)
(0, 4, 81)
(0, 1, 4, 6, 7, 8, 82)
(0, 2, 4, 7, 83)
(0, 1, 3, 5, 7, 8, 84)
(0, 1, 2, 8, 85)
(0, 2, 5, 6, 86)
(0, 1, 5, 7, 87)
(0, 1, 3, 4, 5, 8, 88)
(0, 3, 5, 6, 89)
(0, 2, 3, 5, 90)
(0, 1, 2, 3, 5, 6, 7, 91)
(0, 2, 5, 6, 92)
(0, 2, 93)
(0, 1, 5, 6, 94)
(0, 1, 2, 4, 5, 6, 95)
(0, 2, 3, 4, 6, 7, 96)
(0, 6, 97)
(0, 1, 2, 3, 4, 7, 98)
(0, 4, 5, 7, 99)
(0, 2, 7, 8, 100)
(0, 1, 6, 7, 101)
(0, 3, 5, 6, 102)
(0, 2, 3, 4, 5, 7, 103)
(0, 2, 3, 4, 5, 6, 8, 9, 104)
(0, 1, 2, 4, 5, 6, 105)
(0, 1, 5, 6, 106)
(0, 1, 2, 3, 5, 7, 107)
(0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 108)
(0, 2, 4, 5, 109)
(0, 1, 4, 6, 110)
(0, 2, 4, 7, 111)
(0, 1, 2, 3, 6, 8, 112)
(0, 2, 3, 5, 113)
(0, 2, 3, 6, 7, 8, 114)
(0, 1, 2, 3, 5, 7, 115)
(0, 2, 5, 6, 116)
(0, 1, 2, 5, 117)
(0, 2, 5, 6, 118)
(0, 8, 119)
(0, 1, 2, 5, 6, 7, 120)
(0, 1, 5, 8, 121)
(0, 1, 2, 6, 122)
(0, 2, 123)
(0, 5, 6, 7, 124)
(0, 1, 2, 3, 5, 7, 125)
(0, 2, 4, 7, 126)
(0, 1, 127)
(0, 1, 2, 7, 128)

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

LFSR Notation

LFSRs are often specified by the polynomial

$$P(x) = x^m + p_{m-1}x^{m-1} + \cdots + p_1 x + p_0.$$

Maximum-Length Sequences

Only LFSRs associated with **primitive polynomials** generate maximum-length sequences.

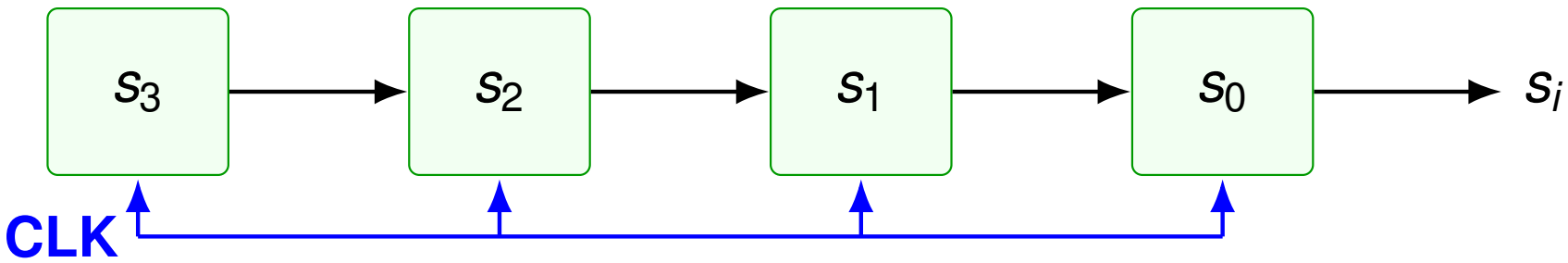
For an m-stage LFSR, the maximum possible period is

$$\boxed{2^m - 1}.$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Exercise 1: LFSR Analysis

Consider an LFSR of degree $m = 4$ with feedback coefficients $p_3 = p_2 = 0, \qquad p_1 = p_0 = 1.$

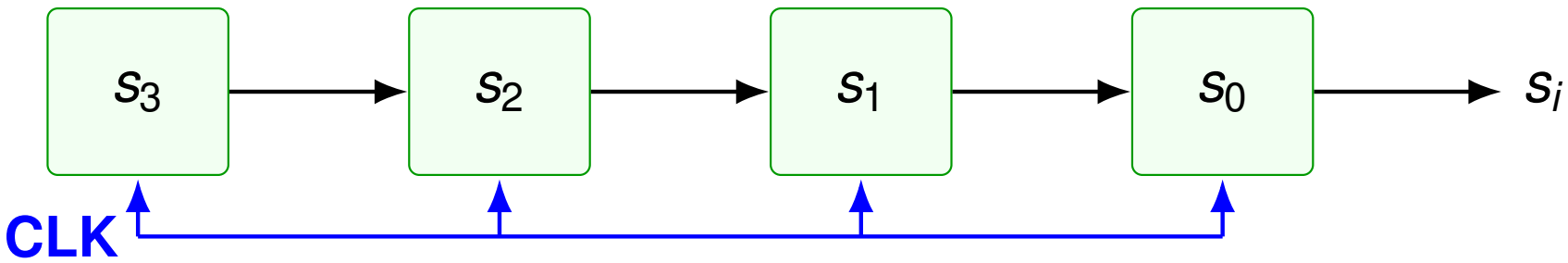


1. Write the recurrence relation.
2. Choose a non-zero initial state.
3. Generate the complete output sequence.
4. Determine the period of the LFSR.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Exercise 2: LFSR Analysis

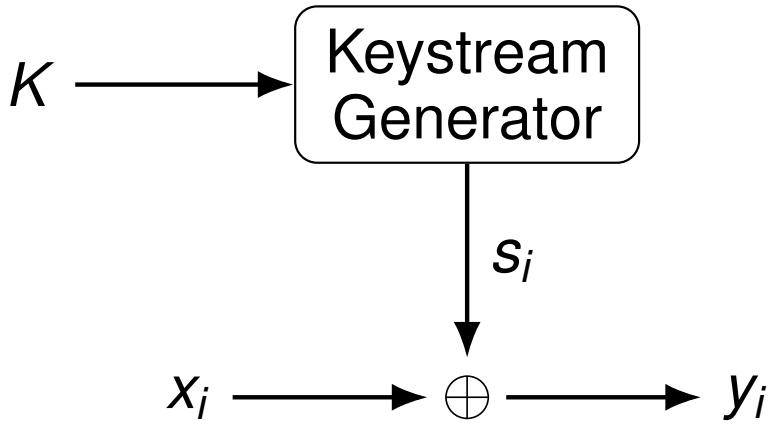
Consider an LFSR of degree $m = 4$, with all feedback coefficients active: $p_3 = p_2 = p_1 = p_0 = 1$.



1. Write the recurrence relation.
2. Select a non-zero initial state.
3. Generate the output sequence.
4. Determine its period.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Attacker's Goal



$$y_i = x_i \oplus s_i \pmod 2$$

Given

- All ciphertext bits y_i
- LFSR degree m
- The first $2m$ plaintext bits $x_0, \ldots, x_{2m-1}$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Attack on an LFSR Stream Cipher: Step 1

Recover the known keystream bits

From $y_i \equiv x_i + s_i \pmod 2$, we obtain $s_i \equiv y_i + x_i \pmod 2$.

Therefore, the attacker can recover

$$\boxed{s_0, s_1, \ldots, s_{2m-1}}$$

for $i = 0, 1, \ldots, 2m-1$.

Key idea

Knowing x_i and y_i reveals the corresponding keystream bit s_i.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Attack on an LFSR Stream Cipher: Step 2

Goal

Use the recovered keystream bits $s_0, s_1, \ldots, s_{2m-1}$ to determine the unknown feedback coefficients

$$\boxed{p_0, p_1, \ldots, p_{m-1}}$$

of the LFSR.

The LFSR recurrence is

$$s_{m+i} \equiv \sum_{j=0}^{m-1} p_j \, s_{i+j} \pmod 2.$$

Using the known keystream bits, construct equations for

$$i = 0, 1, \ldots, m-1.$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

LFSR Linear System

Using the recovered keystream bits, the LFSR recurrence gives:

$$
\begin{aligned}
i = 0, \quad & s_m && \equiv && p_{m-1}s_{m-1} + \cdots + p_1 s_1 + p_0 s_0 \pmod 2 \\
i = 1, \quad & s_{m+1} && \equiv && p_{m-1}s_m + \cdots + p_1 s_2 + p_0 s_1 \pmod 2 \\
& \vdots && \vdots && \vdots \\
i = m-1, \quad & s_{2m-1} && \equiv && p_{m-1}s_{2m-2} + \cdots + p_1 s_m + p_0 s_{m-1} \pmod 2
\end{aligned}
$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Solving for the Feedback Coefficients

The previous equations form a system of

Matrix Form

$$\begin{bmatrix} s_{m-1} & s_{m-2} & \cdots & s_0 \\ s_m & s_{m-1} & \cdots & s_1 \\ \vdots & \vdots & \ddots & \vdots \\ s_{2m-2} & s_{2m-3} & \cdots & s_{m-1} \end{bmatrix} \begin{bmatrix} p_{m-1} \\ p_{m-2} \\ \vdots \\ p_0 \end{bmatrix} \equiv \begin{bmatrix} s_m \\ s_{m+1} \\ \vdots \\ s_{2m-1} \end{bmatrix} \pmod 2.$$

Goal

Solve the system over $\mathbb{Z}_2$ to recover the LFSR feedback coefficients.

Known-Plaintext Attack Against a Single LFSR

Suppose the attacker knows $2m$ plaintext/ciphertext pairs:

$$(x_i, y_i), \qquad i = 0, \ldots, 2m-1.$$

The keystream is immediately recovered:

$$s_i = x_i \oplus y_i.$$

The recurrence provides m linear equations in the m unknown coefficients

$$p_0, p_1, \ldots, p_{m-1}.$$

Result

The feedback coefficients can be recovered efficiently using Gaussian elimination. A single LFSR is therefore not a secure stream cipher.

AND COMPUTATIONAL SCIENCES

What the LFSR Attack Teaches Us

1. Long period does **not** imply security.
2. Good statistical properties do **not** imply unpredictability.
3. Linear state evolution is vulnerable to algebraic reconstruction.
4. Secure stream ciphers introduce **nonlinearity** and larger internal state.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Security Weakness of LFSRs (cont.)

Main weakness

If an attacker knows $2m$ output bits of an LFSR of degree m, the feedback coefficients p_i can be recovered by solving a system of linear equations over $\mathbb{Z}_2$.

Once the coefficients are known:

1. Reconstruct the LFSR.
2. Load any m consecutive known output bits.
3. Clock the LFSR to reproduce the complete output sequence.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Security Weakness of LFSRs (cont.)

Cryptographic consequence

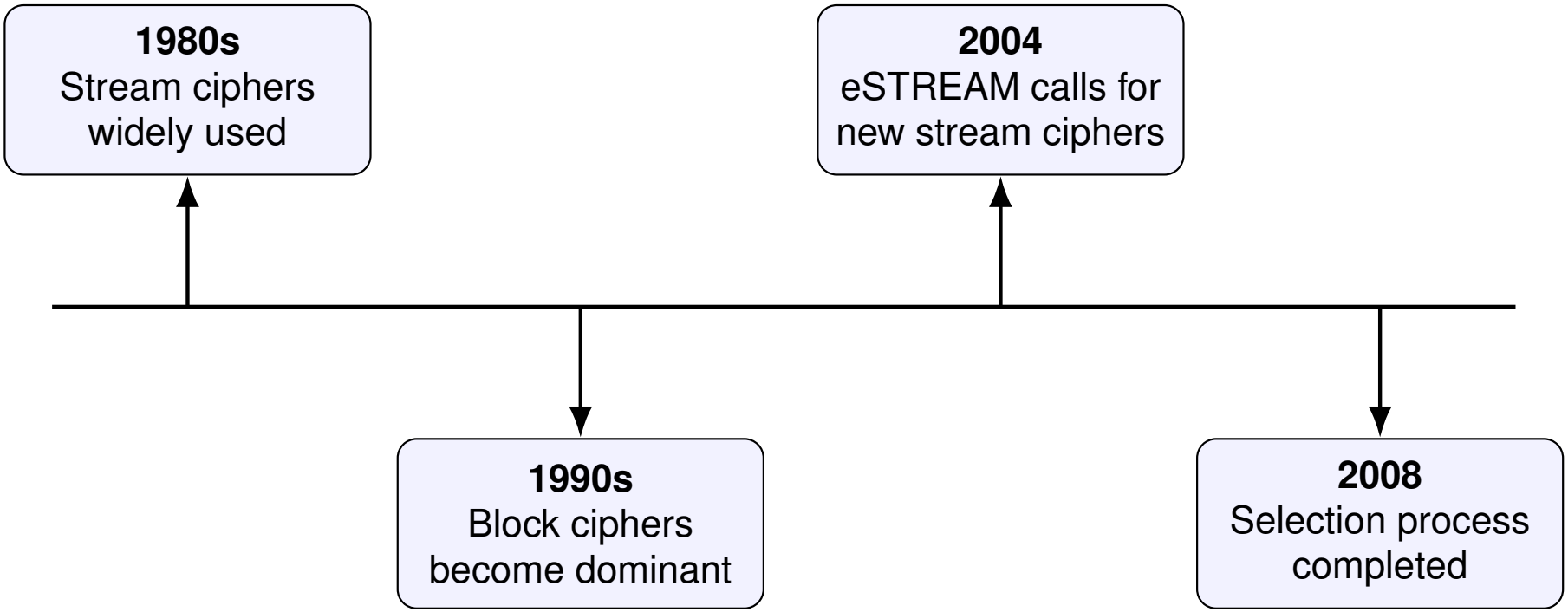
LFSRs may have good statistical properties, but a single LFSR is **cryptographically insecure** because its output is predictable.

Practical solution

Secure stream ciphers combine multiple LFSRs or add nonlinear operations to prevent this reconstruction attack.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Practical Stream Ciphers (cont.)



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Practical Stream Ciphers (cont.)

Why eSTREAM?

Successful attacks against several early stream ciphers motivated a new European effort to design and evaluate stronger algorithms.

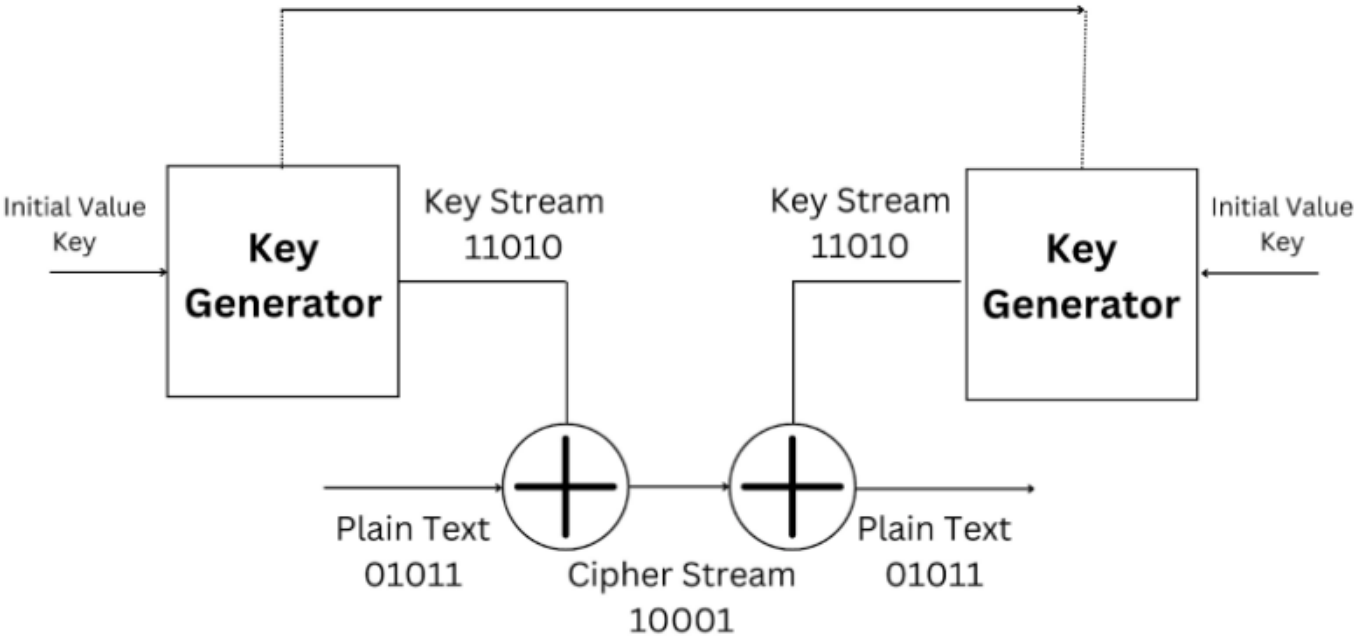
Selected Profiles

- **Profile 1:** high-throughput software ciphers — Salsa20 / ChaCha.
- **Profile 2:** hardware-oriented ciphers with low gate count and low power consumption — Trivium.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Trivium

Trivium: Internal Structure



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

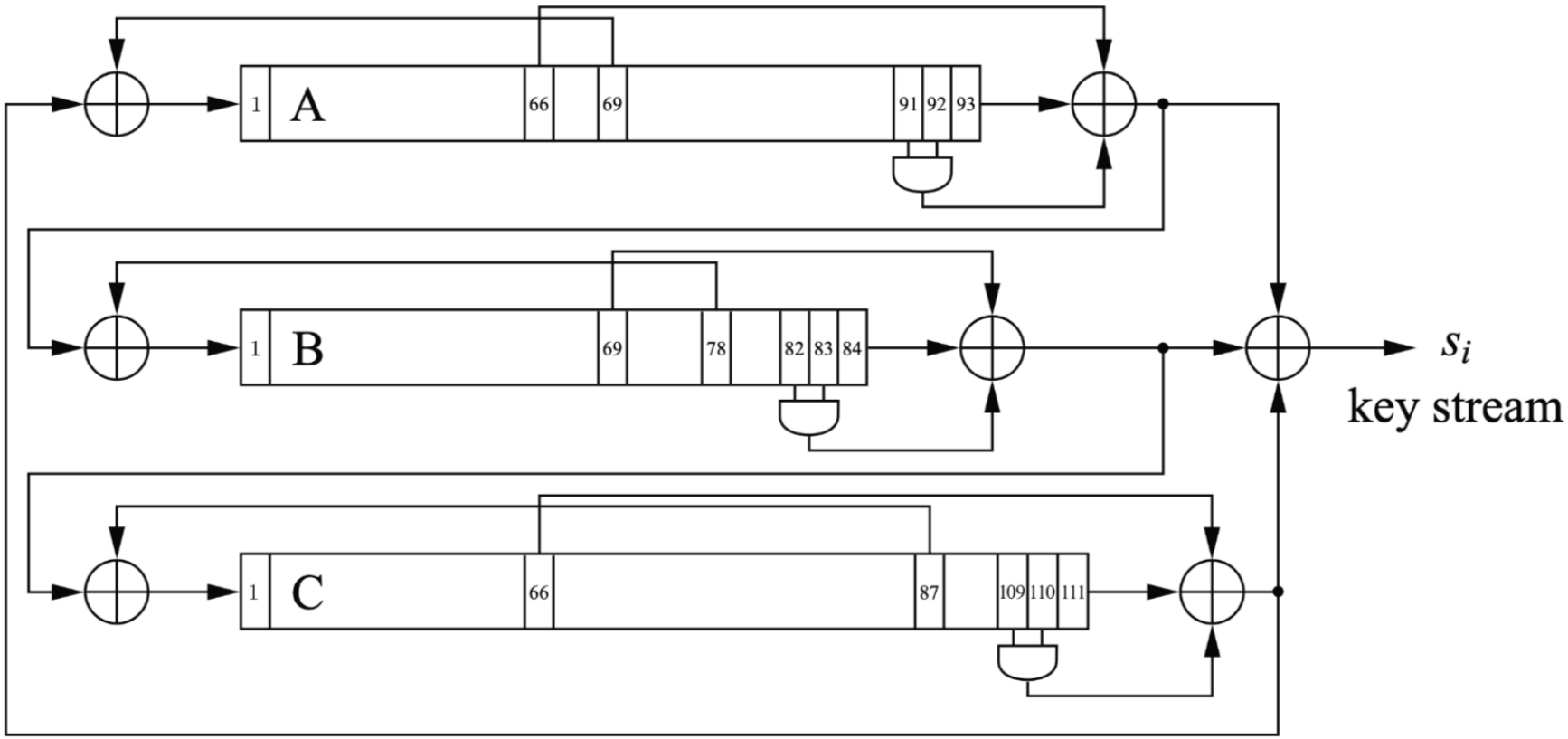
Trivium: Design Overview

Trivium is a hardware-oriented synchronous stream cipher based on three nonlinear shift-register components.

Register	Length	Role
A	93	state / nonlinear feedback
B	84	state / nonlinear feedback
C	111	state / nonlinear feedback
Total	**288 bits**	

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Trivium: Internal Structure



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Trivium Initialization

Trivium uses:

- an 80-bit secret key;
- an 80-bit initialization vector (IV);
- a 288-bit internal state.

After loading the key and IV, the cipher performs

$$4 \times 288 = 1152$$

initialization clocks without producing keystream output.

Key Point

The warm-up phase diffuses the key and IV throughout the internal state before encryption begins.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Why Trivium Is Different from a Single LFSR

- Three interacting state registers are used.
- Feedback includes nonlinear AND operations.
- The keystream combines multiple internal state bits.
- The simple linear reconstruction attack against a single LFSR no longer applies directly.

Design principle

Efficiency is retained, while nonlinearity is introduced to resist linear prediction.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Security Hierarchy

Generator / Cipher	Main property	Cryptographic status
TRNG	Physical randomness	Key/seed generation
PRNG	Deterministic statistics	Not necessarily secure
CSPRNG	Unpredictability	Suitable for keystreams
OTP	Truly random one-use key	Perfect secrecy
Single LFSR	Long linear sequence	Insecure alone
Trivium	Nonlinear 288-bit state	Practical stream cipher

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Common Security Mistakes

- Reusing a keystream or OTP key.
- Using a general-purpose PRNG instead of a CSPRNG.
- Assuming a long period implies cryptographic security.
- Ignoring IV/nonce requirements.
- Designing a keystream generator whose state can be reconstructed from output.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Lessons Learned

1. Stream ciphers encrypt by combining plaintext with a keystream using XOR.
2. The keystream generator determines the security of the cipher.
3. CSPRNGs require unpredictability, not merely good statistics.
4. OTP achieves perfect secrecy but requires a random, message-length, one-use key.
5. Single LFSRs are efficient but insecure because of linearity.
6. Practical designs such as Trivium introduce nonlinear state evolution.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Discussion Questions

1. Why does XOR with a random bit hide the plaintext bit?
2. Why can a PRNG pass statistical tests and still be cryptographically insecure?
3. Why is key reuse catastrophic for the One-Time Pad?
4. Why are $2m$ known keystream bits enough to attack an LFSR of degree m?
5. What security property does Trivium add that a single LFSR lacks?

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problems

Problem 1: Stream Ciphers

Consider the plaintext

$$\mathbf{x} = 1011\,0010$$

and the key stream

$$\mathbf{s} = 1100\,1010.$$

1. Compute the ciphertext $\mathbf{y}$.
2. Decrypt $\mathbf{y}$ using the same key stream.
3. Verify that
$$(\mathbf{x} \oplus \mathbf{s}) \oplus \mathbf{s} = \mathbf{x}.$$
4. Explain why XOR can be used for both encryption and decryption.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 2: Key-Stream Recovery

The following plaintext and ciphertext are known:

$$\mathbf{x} = 1010\,1100\,0110$$

$$\mathbf{y} = 0111\,1001\,1100.$$

1. Recover the key stream **s**.
2. Write the relationship used to recover each bit s_i.
3. Verify your result by encrypting the plaintext again.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 3: LFSR Sequence Generation

Consider a 3-bit LFSR with state

$$(S_2, S_1, S_0)$$

and feedback rule

$$f = S_2 \oplus S_0.$$

The initial state is

$$(S_2, S_1, S_0) = (1, 0, 1).$$

1. Compute the first 10 output bits.
2. Complete the sequence of internal states.
3. Determine the period of the LFSR.
4. Does the LFSR visit all nonzero 3-bit states?
5. What happens if the initial state is 000?

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 4: Determine the LFSR Feedback

An LFSR generates the following key stream:

$$\mathbf{s} = 1\,0\,1\,1\,1\,0\,0\,1\,0\,1\,1\,1\,0\,0\,1.$$

Assume that the generator has degree

$$m = 3$$

and satisfies the recurrence

$$s_{i+3} = p_2 s_{i+2} \oplus p_1 s_{i+1} \oplus p_0 s_i,$$

where

$$p_0, p_1, p_2 \in \{0, 1\}.$$

1. Use the first output bits to construct a system of equations over $\mathrm{GF}(2)$.
2. Determine the coefficients
$$p_2,\ p_1,\ p_0.$$
3. Write the corresponding connection polynomial.
4. Determine the initial state of the LFSR.
5. Draw the LFSR circuit associated.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 5: Maximum-Length LFSR

Consider the primitive polynomial for maximum-length LFSR :

$$(0, 2, 3, 4, 8).$$

1. Write the corresponding primitive polynomial $P(x)$.
2. Determine the number of stages required for the LFSR.
3. Draw the corresponding LFSR circuit, assuming that the register shifts from left to right.
4. Determine the maximum period of the LFSR.
5. How many different non-zero states can the LFSR generate before repeating the initial state?
6. What happens if the initial state is

   $$00000000?$$

7. If two different non-zero seeds are used, will both LFSRs generate the same set of states? Explain.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 6: Maximum-Length 24-bit LFSR

Consider a 24-bit LFSR defined by the primitive polynomial

$$P(x) = x^{24} + x^4 + x^3 + x + 1.$$

The LFSR is initialized with a non-zero 24-bit seed.

1. Determine the maximum possible period of the LFSR.
2. How many different non-zero 24-bit states can be generated before the sequence repeats?
3. Explain what happens if the initial seed is

   $$000000000000000000000000.$$

4. Suppose the LFSR is initialized with two different non-zero seeds,

   $$S_A \neq S_B.$$

   Will both seeds generate the same set of states? Explain your answer.
5. Does the LFSR generate the states in increasing numerical order?

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 7: LFSR Analysis

We will analyze a pseudorandom number sequence generated by an LFSR characterized by:

$$(p_2, p_1, p_0) = (1, 0, 1)$$

1. What is the sequence generated from the initialization vector

$$(s_2, s_1, s_0) = (1, 0, 0)?$$

2. What is the sequence generated from the initialization vector

$$(s_2, s_1, s_0) = (0, 1, 1)?$$

3. How are the two sequences related?

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 8: Attack on an LFSR-Based Stream Cipher

A stream cipher uses a single **LFSR** as its key-stream generator. The LFSR has degree:

$$256$$

1. How many plaintext/ciphertext bit pairs are required to launch a successful attack?
2. Describe the attack step by step and derive the equations that must be solved.
3. What is the key in this system?
   Why is it not a good idea to use the initial contents of the LFSR as the key, or as part of the key?

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 9: LFSR Sequences

LFSRs can be classified according to their connection polynomial:

- **Primitive:** generates a maximum-length sequence.
- **Irreducible but not primitive:** sequence length is independent of the initial state.
- **Reducible:** sequence length depends on the initial state.

For each polynomial, determine **all sequences** generated by the corresponding LFSR:

$$P_1(x) = x^4 + x + 1,$$
$$P_2(x) = x^4 + x^2 + 1,$$
$$P_3(x) = x^4 + x^3 + x^2 + x + 1.$$

1. Draw the corresponding LFSR.
2. Determine all generated sequences and their periods.
3. Classify each polynomial as **primitive**, **irreducible**, or **reducible**.
4. Verify that the sum of all sequence lengths is

$$2^m - 1 = 2^4 - 1 = 15.$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 10: Known-Plaintext Attack on an LFSR

We conduct a known-plaintext attack on an LFSR-based stream cipher.

The known plaintext is:

$$\mathbf{x} = 1001\ 0010\ 0110\ 1101\ 1001\ 0010\ 0110$$

By tapping the communication channel, we observe the ciphertext:

$$\mathbf{y} = 1011\ 1100\ 0011\ 0001\ 0010\ 1011\ 0001$$

1. Determine the degree m of the LFSR key-stream generator.
2. Determine the initialization vector (IV) of the LFSR.
3. Determine the feedback coefficients

$$p_0, p_1, \ldots, p_{m-1}.$$

4. Draw the corresponding LFSR circuit and verify that it generates the recovered key-stream sequence.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 11-A: LFSR-Based Stream Cipher Attack

We want to perform an attack on an **LFSR-based stream cipher**.

Each uppercase letter and the digits 0, 1, 2, 3, 4, 5 are represented by a **5-bit vector** according to:

$$A \leftrightarrow 0 = 00000_2,$$
$$\vdots$$
$$Z \leftrightarrow 25 = 11001_2,$$
$$0 \leftrightarrow 26 = 11010_2,$$
$$\vdots$$
$$5 \leftrightarrow 31 = 11111_2.$$

We know the following information:

- The degree of the LFSR is
$$m = 6.$$
- Every message starts with the known header
$$\texttt{WPI}.$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 11-B: LFSR-Based Stream Cipher Attack

The following ciphertext is observed on the channel:

`J5A0EDJ2B`

The fourth character is the digit `0`.

1. What is the **initialization vector** of the LFSR?
2. What are the **feedback coefficients** of the LFSR?
3. Write a program that generates the complete key stream and recover the entire plaintext.
4. Where does the object mentioned after `WPI` live?
5. What type of cryptanalytic attack was performed?

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Problem 12: Testing Trivium with CrypTool 2

Analyze the operation of the Trivium by generating a key stream and applying it to message encryption and decryption.

Using **CrypTool 2**:

1. Create a plaintext message of at least 32 characters.
2. Configure Trivium with:
   - an 80-bit secret key,
   - an 80-bit IV.
3. Generate the key stream.
4. Encrypt the plaintext.
5. Decrypt the ciphertext using the same key and IV.

Verify:

$$P \oplus S = C$$

and

$$C \oplus S = P$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Cryptography

Stream Ciphers

Cristhian Iza

School of Mathematical and Computational Sciences
Yachay Tech University

August 26, 2026

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES